

Progress Report

Day 11 — Audit History API & Pagination

Task Management MERN Stack Platform

Shafin Nigamana | 29 May 2026

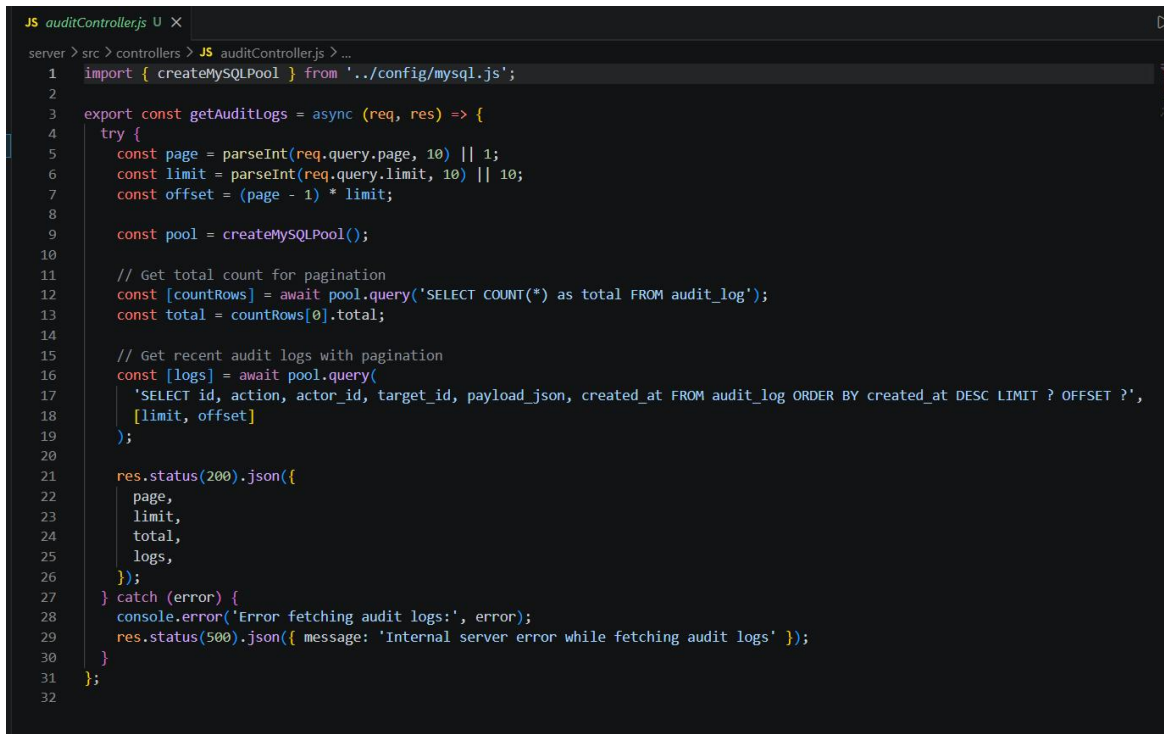
1. Day 11 Objective

The goal for Day 11 was to expose audit history through a backend API, implement pagination support for audit records, and verify the complete audit logging workflow using MySQL and Postman.

2. Tasks Completed

Audit Controller Implementation

- Created auditController.js inside server/src/controllers
- Implemented getAuditLogs() function for retrieving audit history from MySQL
- Configured response structure for paginated audit data retrieval



```
JS auditController.js U X
server > src > controllers > JS auditController.js > ...
1 import { createMySQLPool } from '../config/mysql.js';
2
3 export const getAuditLogs = async (req, res) => {
4   try {
5     const page = parseInt(req.query.page, 10) || 1;
6     const limit = parseInt(req.query.limit, 10) || 10;
7     const offset = (page - 1) * limit;
8
9     const pool = createMySQLPool();
10
11     // Get total count for pagination
12     const [countRows] = await pool.query('SELECT COUNT(*) as total FROM audit_log');
13     const total = countRows[0].total;
14
15     // Get recent audit logs with pagination
16     const [logs] = await pool.query(
17       'SELECT id, action, actor_id, target_id, payload_json, created_at FROM audit_log ORDER BY created_at DESC LIMIT ? OFFSET ?',
18       [limit, offset]
19     );
20
21     res.status(200).json({
22       page,
23       limit,
24       total,
25       logs,
26     });
27   } catch (error) {
28     console.error('Error fetching audit logs:', error);
29     res.status(500).json({ message: 'Internal server error while fetching audit logs' });
30   }
31 }
32
```

Figure 2.1 — Audit controller implementation for retrieving audit history.

Audit Route Setup

- Created auditRoutes.js inside backend routes structure
- Implemented GET /api/audit endpoint
- Protected audit endpoint using existing JWT authentication middleware



```
server > src > routes > JS auditRoutes.js > ...
1  import express from 'express';
2  import { getAuditLogs } from '../controllers/auditController.js';
3  import { protect } from '../middleware/auth.js';
4
5  const router = express.Router();
6
7  // Get audit logs history (protected)
8  router.get('/', protect, getAuditLogs);
9
10 export default router;
11
```

Figure 2.2 — Protected audit route implementation.

Audit API Integration

- Registered /api/audit route inside Express application
- Integrated MySQL audit history retrieval with backend API layer
- Verified successful communication between Express backend and MySQL audit table

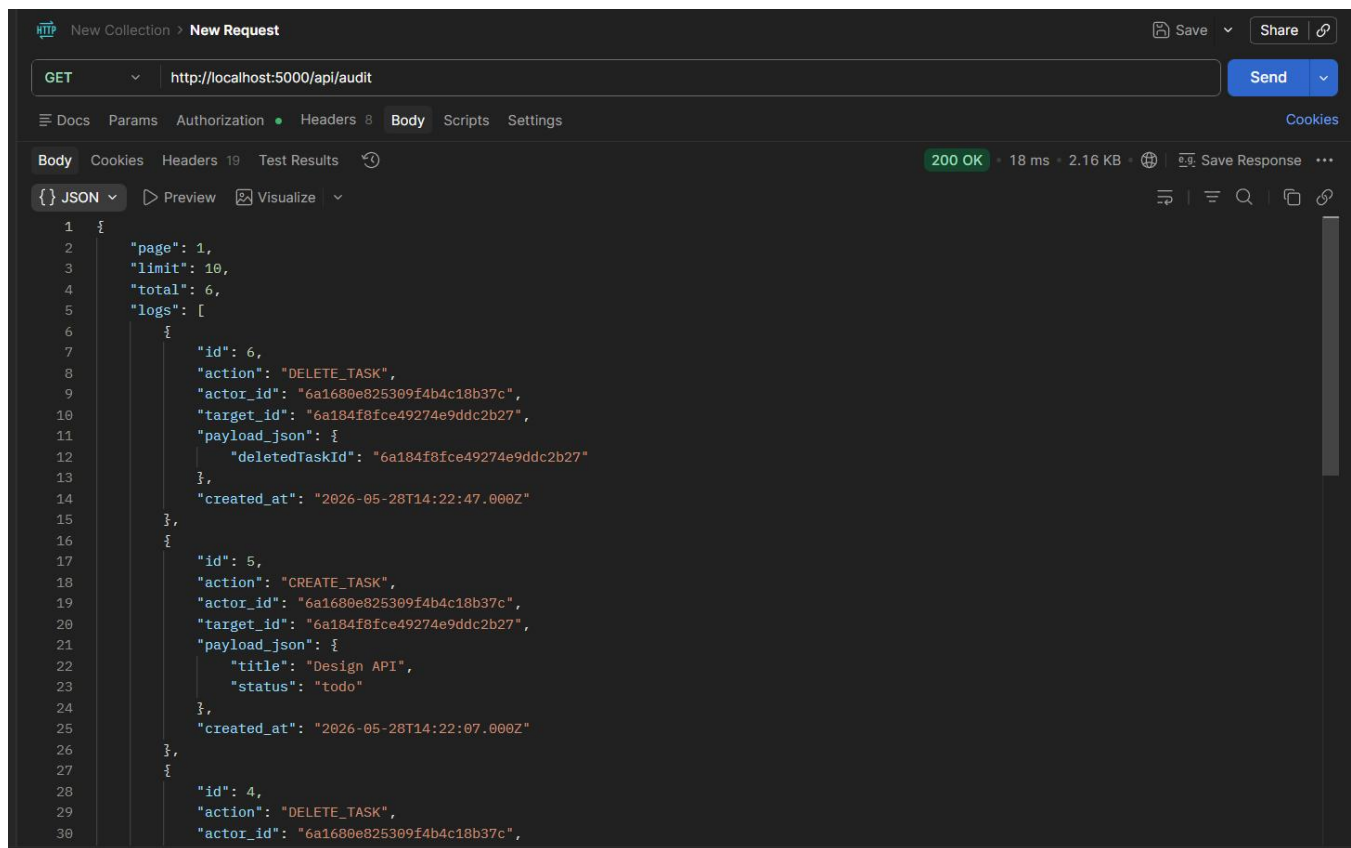


Figure 2.3 — Audit history retrieved successfully through API endpoint.

Pagination Implementation

- Implemented pagination support using:
 - page
 - limit
- Configured SQL LIMIT and OFFSET queries for audit record retrieval
- Verified paginated responses using Postman

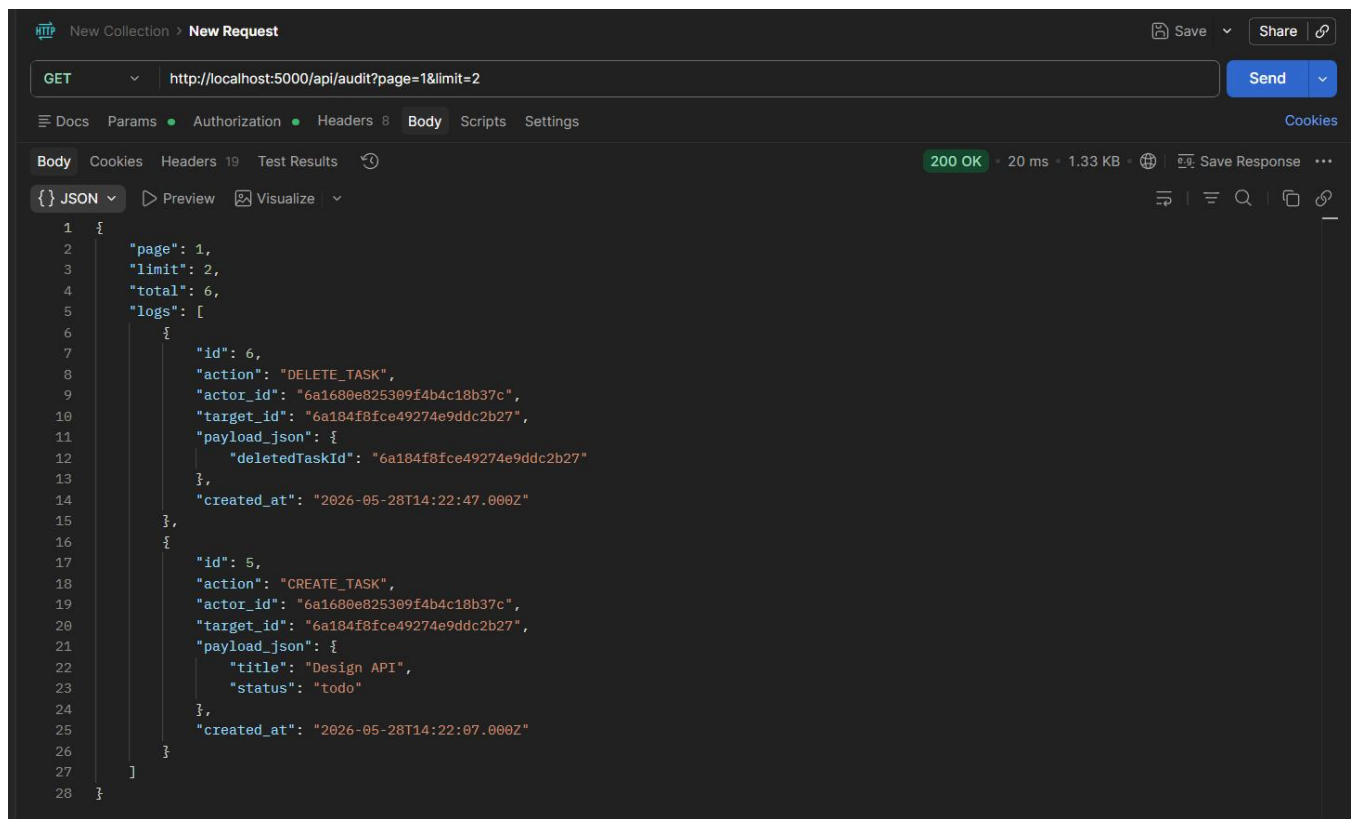


Figure 2.4 — Audit history pagination verified successfully.

Authentication Verification

- Verified audit endpoint requires valid JWT authentication
- Confirmed unauthorized requests are blocked correctly using authentication middleware

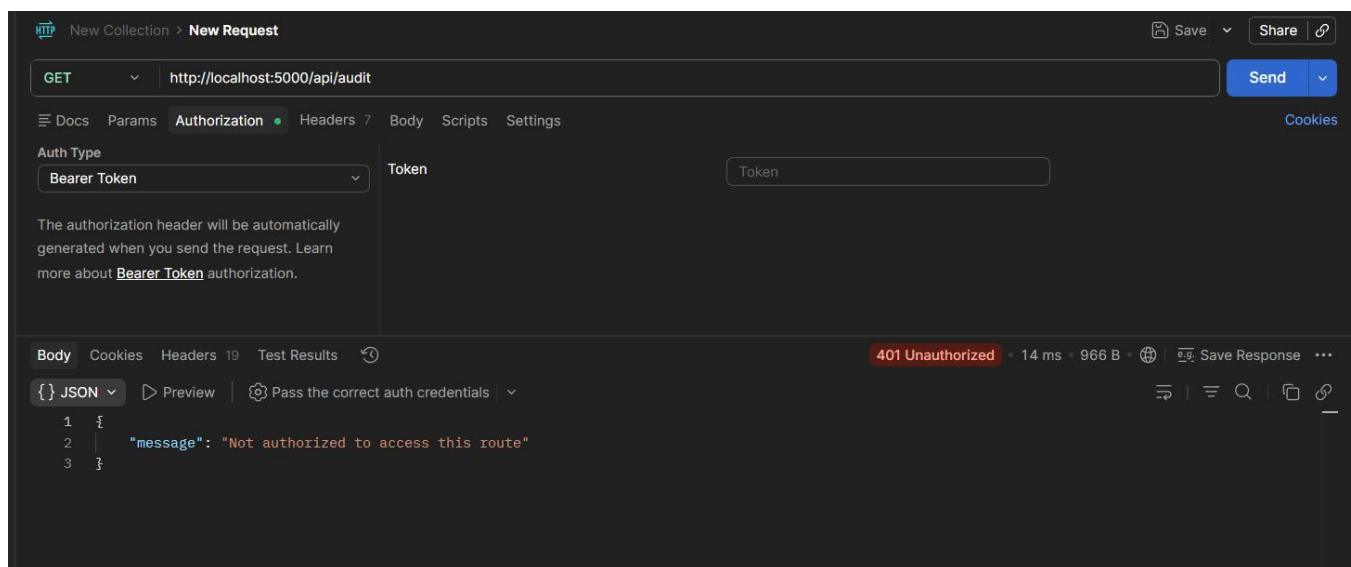


Figure 2.5 — Unauthorized audit access blocked successfully.

Repository Update

- Committed audit endpoint implementation changes using Git workflow
- Pushed latest backend milestone updates to GitHub repository

```

$ git status
On branch feature/task-module
Your branch is up to date with 'origin/feature/task-module'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   server/src/app.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        server/src/controllers/auditController.js
        server/src/routes/auditRoutes.js

no changes added to commit (use "git add" and/or "git commit -a")

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/task-module)
$ git add .
warning: in the working copy of 'server/src/app.js', LF will be replaced by CRLF the next time Git touches it

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/task-module)
$ git commit -m "feat: implement audit history API with pagination"
[feature/task-module 10c40b6] feat: implement audit history API with pagination
3 files changed, 43 insertions(+)
create mode 100644 server/src/controllers/auditController.js
create mode 100644 server/src/routes/auditRoutes.js

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/task-module)
$ git push
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 1.33 KiB | 194.00 KiB/s, done.
Total 9 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 6 local objects.
To github.com:ShafinNigamana/task-management-mern.git
4bc1b48..10c40b6 feature/task-module -> feature/task-module

```

Figure 2.6 — Day 11 audit history API implementation pushed to GitHub.

3. Next Steps

- Update README with latest API endpoints
- Review backend structure and documentation
- Begin frontend planning and route design
- Prepare dashboard and authentication UI architecture

4. Conclusion

Day 11 focused on implementing the Audit History API and pagination support. Audit records stored in MySQL can now be retrieved through protected backend endpoints, providing a complete audit tracking workflow for Task operations.

